

Corso di Laurea in Informatica

Linguaggi

Esame del 4 Febbraio 2020

- Si ricorda che ogni risposta va giustificata ed ogni concetto spiegato nel corso che viene citato va definito. Per ogni esercizio si indica tra parentesi il suo valore in 30-esimi.
- Per svolgere tutti gli esercizi esclusi gli ultimi due si hanno a disposizione 2h (questa parte va **OBBLIGATORIAMENTE** consegnata al termine delle 2h).
- Per svolgere **UNO** degli ultimi due esercizi si hanno a disposizione ulteriori 20 minuti
- Gli iscritti dal 2018/19 in poi, che non hanno consegnato il progetto **NON POSSONO** rispondere a nessuno degli ultimi due quesiti
- Gli iscritti ad un anno precedente al 2018/19 che hanno consegnato il progetto possono rispondere **ESCLUSIVAMENTE** alla domanda sul progetto.

1. (4) Definire intuitivamente e formalmente (mediante definizione semantica) cosa è un interprete. Descrivere e spiegare poi la struttura di un interprete (come pseudo-codice o come flow-chart).
2. (2) Dimostrare per induzione che $\forall n \in \mathbb{N}. n > 2$ si ha che $n^2 > 2n + 1$.

3. (5) Si consideri il programma sulla destra. Si dica cosa viene calcolato dall'assegnamento in caso di scoping statico (mostrando come vengono calcolati i link statici e come vengono risolti riferimenti non locali) e in caso di scoping dinamico (mostrando l'evoluzione della tabella centrale dei riferimenti (CRT) e come vengono risolti riferimenti non locali).

```
{int a;
void fun1(){
    a = a + 2;}
void fun2(){
    int a = 5;
    void fun3(int b){
        int a = b + 4;}
        fun1();}
    fun1();
    fun3(4);}
a=4;
fun2(); }
```

4. (6) Spiegare dove e perché nel programma è necessario parlare di regole di scoping, e dove (e perché) anche di regole di binding. Si dica quale risulta essere il valore finale di z nelle varie situazioni, ovvero con: (1) scoping statico; (2) scoping dinamico e deep-binding; (3) shallow-binding (e scoping dinamico).

```
int x = 3; int y = 2;
int fun1 (int in){
    return x + y + in;
}
void fun2 (int h(int b)){
    int y = 4; int x = 4;
    return h(5) + x;
}
{int x = 6; int y = 3;
int z = fun2(fun1); }
```

5. (6) Definire brevemente il concetto di ricorsione e di ricorsione in coda. Si consideri il programma ricorsivo a destra e si descriva che cosa calcola. Trasformare quindi tale programma in un programma ricorsivo in coda. Mostrare poi il funzionamento dell'algoritmo fornito sulla lista (3, 6, 9).

```
lista function (lista list){
    if (length(list) = 0) then return ();
    else return concat(2 + car(list), function(cdr(list)));
}
```

NOTA: Data una lista $list$, $length(list)$ restituisce la sua lunghezza, $car(list)$ restituisce il primo elemento, $cdr(list)$ restituisce tutta la lista tranne il primo elemento, $concat$ concatena due liste (se un elemento non è una lista, $concat$ lo converte prima in lista). $()$ è la lista vuota.

6. (4) Siano $\rho = [x \mapsto true, y \mapsto l_y]$ un ambiente dinamico con l_y locazione di tipo bool, e $\sigma = [l_y \mapsto false]$ una memoria. Mostrare le derivazioni della semantica dinamica per il comando $y := x$ **and** y nell'ambiente ρ a partire dalla memoria σ ($\rho \vdash \langle x := x \text{ and } y, \sigma \rangle \rightarrow \dots$).

Esercizi per i quali si hanno ulteriori 20 min a disposizione

1. (4) **[Esclusivamente per chi ha consegnato il progetto, a questa domanda si pu rispondere ad un solo appello e la valutazione rimane valida e immodificabile fino all'appello di febbraio 2021 escluso]**
 - a Si spieghi come è stata modificata la grammatica Imp.g4 per implementare il costrutto 'for' (in particolare, si commenti quali categorie sintattiche sono state modificate/estese/introdotte e perché).
 - b Si spieghi, a parole e/o in pseudocodice, come è stata implementata la semantica dinamica del costrutto 'for'. In particolare, si commenti come l'implementazione realizzata rifletta la semantica intuitiva del ciclo 'for'.
 - c Si commenti dal punto di vista pratico e di efficienza la differenza tra una implementazione iterativa ed una ricorsiva del costrutto 'for' (suggerimento: si ragioni sul caso di un programma che diverge).
2. (3) **[Esclusivamente per iscritti a A.A. strettamente precedenti al 2018/19]** Definire semantica statica e dinamica della seguente nuova espressione, con la semantica intuitiva di valere e_1 se e_0 è vera, e_2 altrimenti.

$$e ::= \text{if } e_0 \text{ then } e_1 \text{ else } e_2$$